CSCI-665 Software Engineering · AI-ENABLED SOFTWARE ENGINEERING

MiniDash: Ship a Feature with an AI Pair

Individual assignment on AI-assisted code creation and feature development

Item	Details
Course topics	Lecture 4 (Architecture & Design) and Lecture 5 (AI-Assisted Code Creation & Feature Development)
Format	Individual work. Discussing ideas with classmates is fine; your repository, log, and reflection must be your own.
Points	**20 points**
Due	**30th Oct '2026** (suggested: two weeks after release)
Submit via	(link to your GitHub repository plus a single PDF of your written pieces)

1. Overview

You are a new engineer at **MiniDash**, a small food-delivery startup that works a little like DoorDash. The core app already lets customers browse restaurants and place orders. Your job is to ship **one new customer-facing feature** end to end, using an AI coding assistant as your pair programmer.

The twist is what we grade. We care less about how much code the AI wrote and more about whether you can **specify, direct, verify, and explain** AI-assisted work. A feature you can defend line by line scores higher than a bigger feature you cannot.

Learning goals

- Turn a feature request into a testable specification (requirements, design, and task list).
- Apply component-level design principles so the new feature fits the existing architecture.
- Use an AI assistant in small, reviewable slices, with prompts that carry context, contracts, and tests.
- Verify AI output: find defects, check dependencies, and write tests that prove something.
- Keep an honest record of AI use and explain every line you commit.

2. The MiniDash Baseline

MiniDash is a small REST API backed by SQLite. Its baseline exposes five endpoints:

Endpoint	Purpose
GET /restaurants	List restaurants with name, cuisine, opening hours, and average prep time
GET /restaurants/{id}/menu	List the menu items (id, name, price) for one restaurant
POST /orders	Place an order: restaurant, items with quantities, delivery address; returns the new order
GET /orders/{id}	Return one order with its items, totals, and status
PATCH /orders/{id}/status	Update the order status (used by restaurant and driver tools)

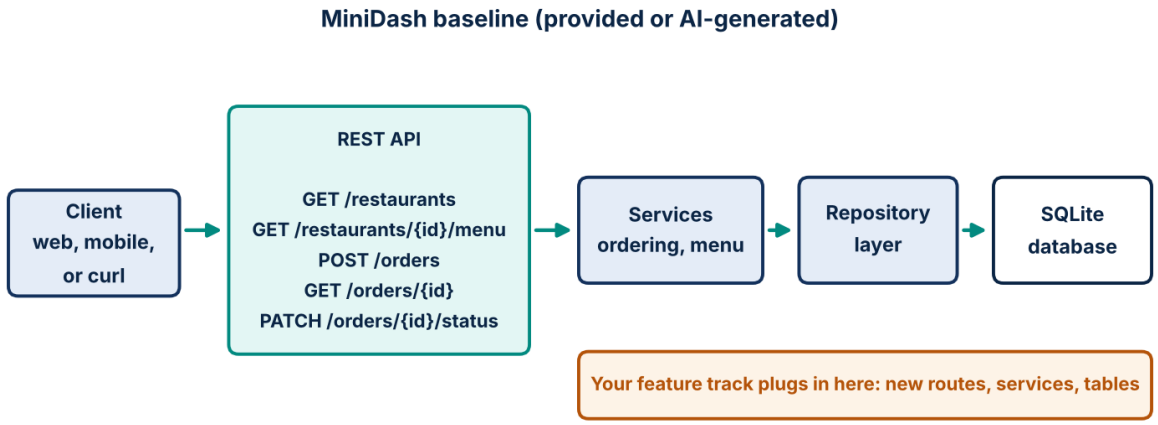


Figure 1. MiniDash baseline architecture. Your feature adds routes, services, and tables.

Getting the baseline. Your instructor will provide a starter repository: _____. If none is provided, you may generate a baseline with an AI tool using the prompt in Appendix C. Either way, the baseline is not graded, but if you generate it with AI, record that in your usage log.

Technology. Use Python (FastAPI) or Node.js (Express) unless your instructor approves another stack. You need an automated test framework (pytest, Jest, or equivalent) and a README that explains how to run the app and the tests.

3. Choose One Feature Track

Pick **one** track. All three are designed to be similar in difficulty.

Track A: Scheduled Orders

A customer can order now for delivery later.

- An order accepts an optional scheduled delivery time. It must be at least **45 minutes** and at most **7 days** ahead; otherwise return a clear validation error.
- Reject a scheduled time when the restaurant is closed.
- A scheduled order is held and released to the restaurant **30 minutes before** its delivery time, moving from SCHEDULED to PLACED exactly once.
- A customer can cancel a scheduled order before it is released.
- All time-dependent logic uses an injected clock so tests do not depend on the real time.

Track B: Group Orders

Friends add items to one shared order, and the host checks out for everyone.

- A host creates a group order for one restaurant and receives a share code.
- Participants join with the code and add or remove their own items until the host locks the order. At most 10 participants; the order expires after 2 hours of inactivity.
- The summary shows each participant's subtotal, and the delivery fee is split evenly with correct rounding (the parts must add up to the whole).
- When the host checks out, the group order becomes a normal MiniDash order.

Track C: Live Order Tracking

Customers can follow an order from placement to delivery.

- Model the status flow PLACED, PREPARING, READY, PICKED_UP, DELIVERED, plus CANCELLED (allowed only before PREPARING). Invalid transitions return an error and change nothing.
- Store a status history with timestamps and expose it at GET /orders/{id}/tracking.
- Return an estimated arrival time computed from the restaurant's prep time and a simple distance model, recomputed when the status changes.
- Provide live updates by polling or by server-sent events, and a small script that simulates a driver advancing an order.

Stretch goals (optional, not required for full marks)

Add a simple front-end page, structured logging, or a rate limit. Stretch work earns credit only if it is verified and documented like the core feature.

4. Required Workflow

Follow the workflow from Lecture 5. Your repository history and your log should make each step visible.

1. **Spec.** Write requirements.md with user stories and acceptance criteria in EARS style (see Appendix A). Include what is out of scope.
2. **Design.** Write design.md: components touched, interfaces and data changes, and at least one Mermaid diagram (component or sequence). Record one real design decision as an ADR. Name the options you considered.
3. **Plan and slice.** Write tasks.md: an ordered list of at least six small slices, each with the test that will verify it.
4. **Implement with AI, one slice at a time.** Use your AI tool for each slice. Commit or open a pull request per slice, with a message that says what changed and how AI was used.
5. **Verify.** Run the tests, linter, and the app. Check every new dependency on its official registry. Review the whole diff for anything outside the slice.
6. **Review and explain.** Before merging a slice, make sure you can explain every line. Log what you checked and what you changed.

5. Deliverables

#	Deliverable	What it must contain
1	Spec package (in /docs)	requirements.md (6+ EARS criteria), design.md with a Mermaid diagram and one ADR, tasks.md with 6+ slices
2	Working code and tests	A GitHub repository with the feature, automated tests that pass, a README with run and test instructions, and at least six slice commits or pull requests with meaningful messages
3	AI usage log (AI_LOG.md)	At least 8 entries using the template in Appendix B. Include at least two entries where you rejected or substantially rewrote the AI's output
4	Reflection (1–2 pages, PDF)	Answers to the five prompts in Section 7, including three specific defects or weaknesses you found in AI output
5	Demo and explain-it-back	A 5-minute demo (live or recorded) of the feature working, followed by a short oral check where you explain code your instructor selects

Suggested layout: /docs, /src, /tests, AI_LOG.md, README.md, and AGENTS.md (your AI rules file).

6. Responsible AI Use

AI assistance is **expected and encouraged** in this assignment. These rules keep the work honest.

- **Disclose.** Every AI-assisted slice appears in your log and in your commit or pull request notes.
- **Own it.** You must be able to explain any code in your repository. If you cannot, treat it as not yours.
- **Verify.** Do not install a package until you have checked it exists on the official registry and is what you intend. Never paste secrets, API keys, or personal data into a prompt.
- **Do not outsource the judgment.** The spec, the design decision, and the reflection are your thinking. You may use AI to improve wording, but note it in the log.
- **Collaboration.** You may discuss approaches with classmates. Sharing code, logs, or written answers is not permitted.

Undisclosed AI use, or code you cannot explain, is treated under the academic integrity policy in the syllabus.

7. Reflection Prompts

1. **Three defects.** Describe three specific problems in AI output (wrong, risky, or wasteful). For each: what it was, how you found it, and how you fixed it.
2. **Where AI helped most and least.** Give one example of each and say why.
3. **Prompting.** What did you change about how you prompted between your first and last slice?

4. **A decision you did not delegate.** Name one design or quality decision you kept for yourself and explain why.
5. **Limits of your evidence.** What is still not verified in your feature, and how would you check it with more time?

8. Free Tools You Can Use

You may use **one or more** AI coding tools at no cost. The options below offer student access or free tiers. Because offers change often, check each vendor's page before relying on a tool, and start verification early because approval can take several days.

Tool	Free access	Good to know
GitHub Copilot (Student plan)	Free for verified students through GitHub Education. Chat, agent mode, and completions inside VS Code and other editors. Models are selected automatically, with a monthly allowance.	Recommended default. Apply for GitHub Education first; if approval is pending, the free Copilot tier gives limited monthly completions and chat.
Kiro (Students tier)	1,000 credits per month for one year, no credit card. Spec-driven editor that creates requirements, design, and task files.	School must be on the eligible list; requires a university email, age 18+, and identity verification. Great match for this assignment's workflow.
Cursor (Hobby plan)	Free plan with no credit card: limited agent requests and tab completions.	Limits can run out quickly. Good for trying an AI-native editor.
Google Antigravity (free individual plan)	No-cost plan with weekly rate limits and a range of agent models.	Limits are not published and may change.
Cline or Continue with a free Gemini API key	Open-source VS Code extensions. A key from Google AI Studio gives free access to Gemini Flash models within rate limits.	No verification needed. Free-tier prompts may be used to improve Google products, so never include private data.
Ollama with Continue (local models)	Free, runs offline on your laptop, no account.	Needs a reasonably capable machine (16 GB RAM or more recommended). Output quality is usually lower than cloud models.

Setup checklist (do this in the first week)

☐ Create or sign in to your GitHub account and apply for GitHub Education with your university email.

☐ Install VS Code (or the editor your chosen tool requires) and your tool of choice.

☐ Run a trivial prompt (for example, "write a function that reverses a string and a test for it") and confirm it works.

☐ Create a short AGENTS.md (or your tool's rules file) with your language, test framework, and three conventions.

☐ Keep a backup plan: set up a second tool from the list in case you hit a usage limit near the deadline.

> **NOTE**
> Paid-only tools are not required for this course. If you have trouble getting any tool working, tell your instructor early. Do not wait until the deadline.

9. Grading Rubric (100 points)

Category	Pts	Full marks look like
Spec and design	20	Criteria are testable and traceable to tasks. The diagram matches the code. The ADR names real alternatives and trade-offs. Scope is clear.
Implementation	25	All core requirements for your track work. Code fits the existing architecture with clear boundaries, no dead code, and sensible error handling.
Tests and verification	20	Tests cover each acceptance criterion and boundary cases, and fail when the code is broken. Dependencies are checked. The test suite passes from a clean clone.
AI usage log and critical review	20	Entries are specific and honest, show verification, and include rejected or rewritten output. Slices are small and prompts carry context.
Reflection and explain-it-back	10	Reflection is specific and self-aware. Oral check shows you understand your code and can justify decisions.
Repository hygiene and communication	5	Meaningful commits, a clear README, and a clean repository with no secrets. Demo is clear and on time.

Common deductions: code you cannot explain; a single large commit; tests that merely repeat the implementation; unchecked dependencies; secrets committed to the repository; AI use missing from the log.

10. Submission Checklist

☐ Repository is accessible to the instructor and contains /docs, /src, /tests, README.md, AI_LOG.md, and your rules file.

☐ Tests pass from a fresh clone using the commands in the README.

☐ At least six slice commits or pull requests, each with a clear message and AI-use note.

☐ AI_LOG.md has at least eight entries, including at least two rejected or substantially rewritten outputs.

☐ Reflection PDF (1–2 pages) answers all five prompts.

☐ Demo is ready (live slot booked or recording link included).

☐ No API keys, passwords, or personal data in the repository history.

Appendix A: Sample EARS Acceptance Criteria

EARS sentences are short, testable requirements. Use them as a pattern, not to copy.

```
WHEN a customer places an order with a scheduled time less than 45 minutes ahead,
  THE SYSTEM SHALL reject it with error SCHEDULE_TOO_SOON.

WHEN a scheduled order is 30 minutes from its delivery time,
  THE SYSTEM SHALL release it to the restaurant kitchen queue exactly once.

IF the restaurant is closed at the requested time,
  THEN THE SYSTEM SHALL reject the order with error RESTAURANT_CLOSED.

WHEN a customer cancels a scheduled order that has not been released,
  THE SYSTEM SHALL mark it CANCELLED and never release it.
```

Appendix B: AI Usage Log Template and Example

Use one entry per significant AI interaction. Copy this table into AI_LOG.md for each entry.

Field	Your entry (example from a Track A student)
Entry # / date	4 / Oct 21, 7:40 pm
Tool, model, mode	GitHub Copilot, agent mode (auto model)
Slice	Slice 3: domain rule validate_schedule()
Prompt (verbatim or summarized)	Pasted the contract and three boundary examples (44, 45, 46 minutes) from requirements.md. Asked for the function and tests only, no new dependencies, only files in /src/orders.
What the AI produced	The function and four tests. It called datetime.now() directly and also added a helper module I did not request.
What I verified	Ran pytest (all passed, but tests used the real clock). Read the full diff. Mutation check: changed 45 to 44 in the rule and a test failed, as expected.
What I changed and why	Injected a clock parameter so tests are deterministic. Deleted the unrequested helper module (scope creep).
Outcome	Modified, then accepted. Committed as "Add 45-minute lead-time rule (AI-assisted, clock injected)".

Appendix C: Prompt for Generating the Baseline (if no starter is provided)

Only use this if your instructor did not provide a starter repository. Review what the AI produces before you build on it.

```
Create a small food-delivery REST API called MiniDash in Python with FastAPI
and SQLite. Tables: restaurants (id, name, cuisine, opens_at, closes_at,
avg_prep_minutes), menu_items (id, restaurant_id, name, price_cents), orders
(id, restaurant_id, status, total_cents, address, created_at) and order_items.
Endpoints: GET /restaurants, GET /restaurants/{id}/menu, POST /orders,
GET /orders/{id}, PATCH /orders/{id}/status. Use a layered structure
(routes, services, repository). Seed 5 restaurants. Add pytest tests for each
endpoint and a README with run instructions. Do not add authentication or
any dependency beyond fastapi, uvicorn, and pytest.
```